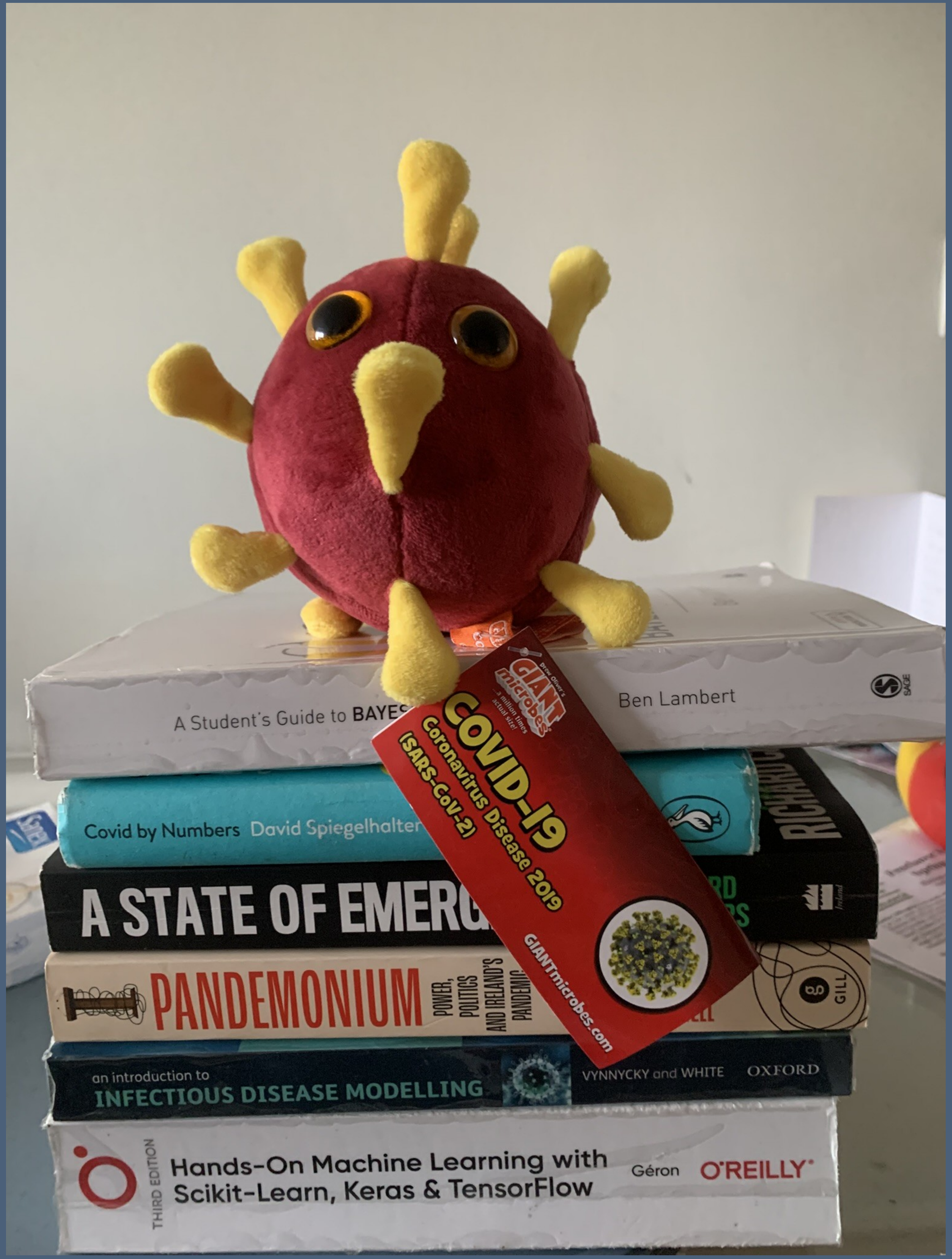
Modelling the COVID Pandemic in Ireland with Bayesian Methods in Python using PyMC and STAN



Peter Nolan
https://github.com/dpnolan/pandemic
databeaker@gmail.com
@dpnolan

Python Ireland
Sunday 16 November 2025
UCD

My Motivations

- Where and when was this photo taken?

https://github.com/dpnolan/pandemic

My Motivations

- In 2020, I was living in Stockholm - Here I am in March at Nobel's tomb

- Family legacy - my grandmother died of TB when my mother was 8, leaving 9 more children under 16

- In February 2006 on David McWilliams' TV show, I said my 5 person software firm had done more pandemic preparation than the Irish government

- The Swedish government were relatively relaxed but data analysts in NGOs and business acted quickly and radically

https://github.com/dpnolan/pandemic

Research Question

- What models can we use for measuring and forecasting the evolution of epidemics?
- What are the tools in Python?

My Approach

- Create and run the models myself
- A test in real-time of the Python solutions
- Nevertheless, our open-source community, academic publications and open data made this project possible, so let's show our support.
- 'All models are wrong, some are useful'. Mine are more likely than most to be wrong, so feel free to comment here or later.

Summary

- Literature review - history of models
- The Susceptible - Infected - Rcovered (SIR) model parameters and dynamics
- Estimating the SIR model parameters
-

https://github.com/dpnolan/pandemic

To give a summary in one slide…

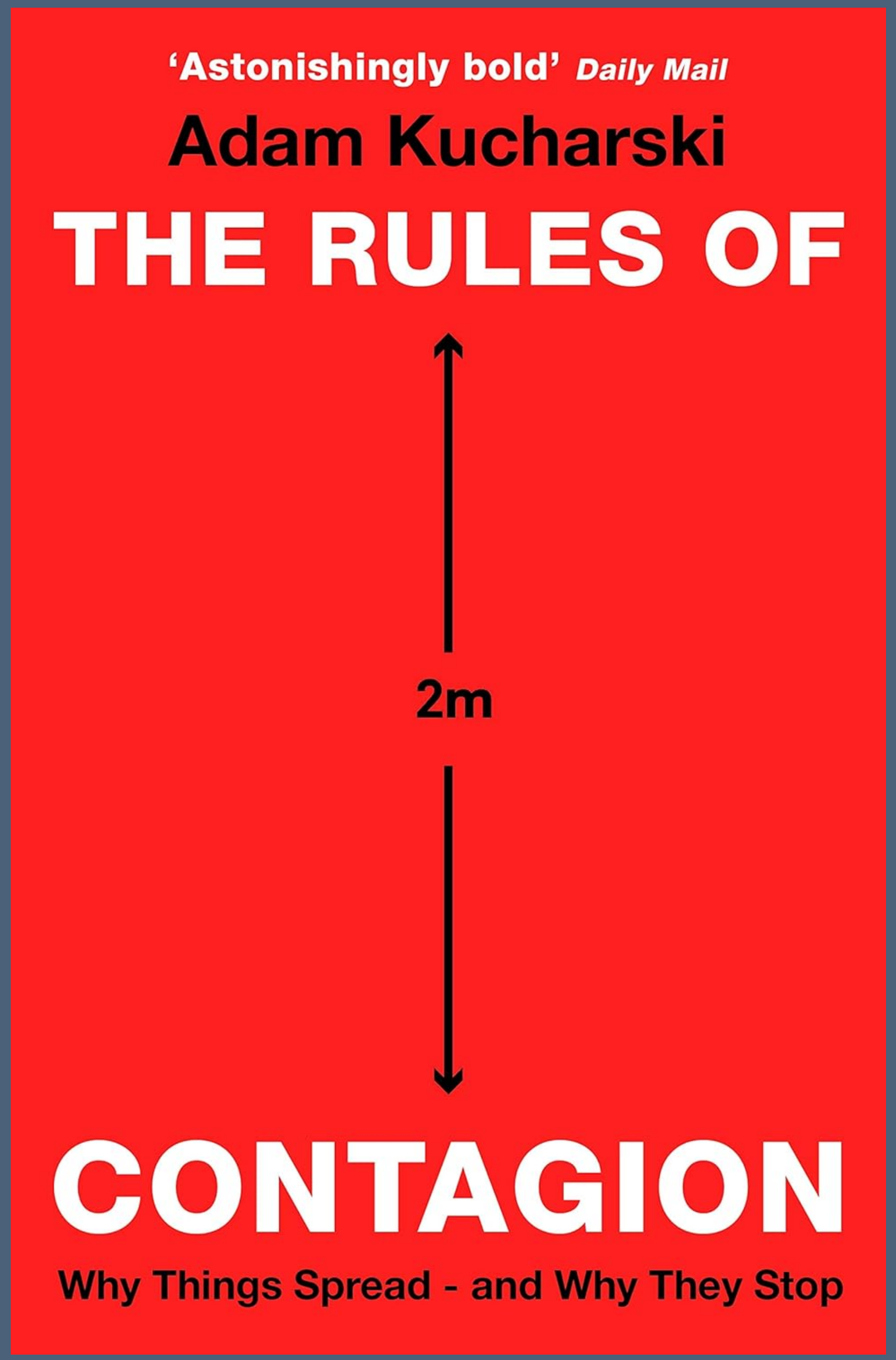
https://github.com/dpnolan/pandemic

To give a summary in one slide...

https://github.com/dpnolan/pandemic

Literature Survey - Popular Science

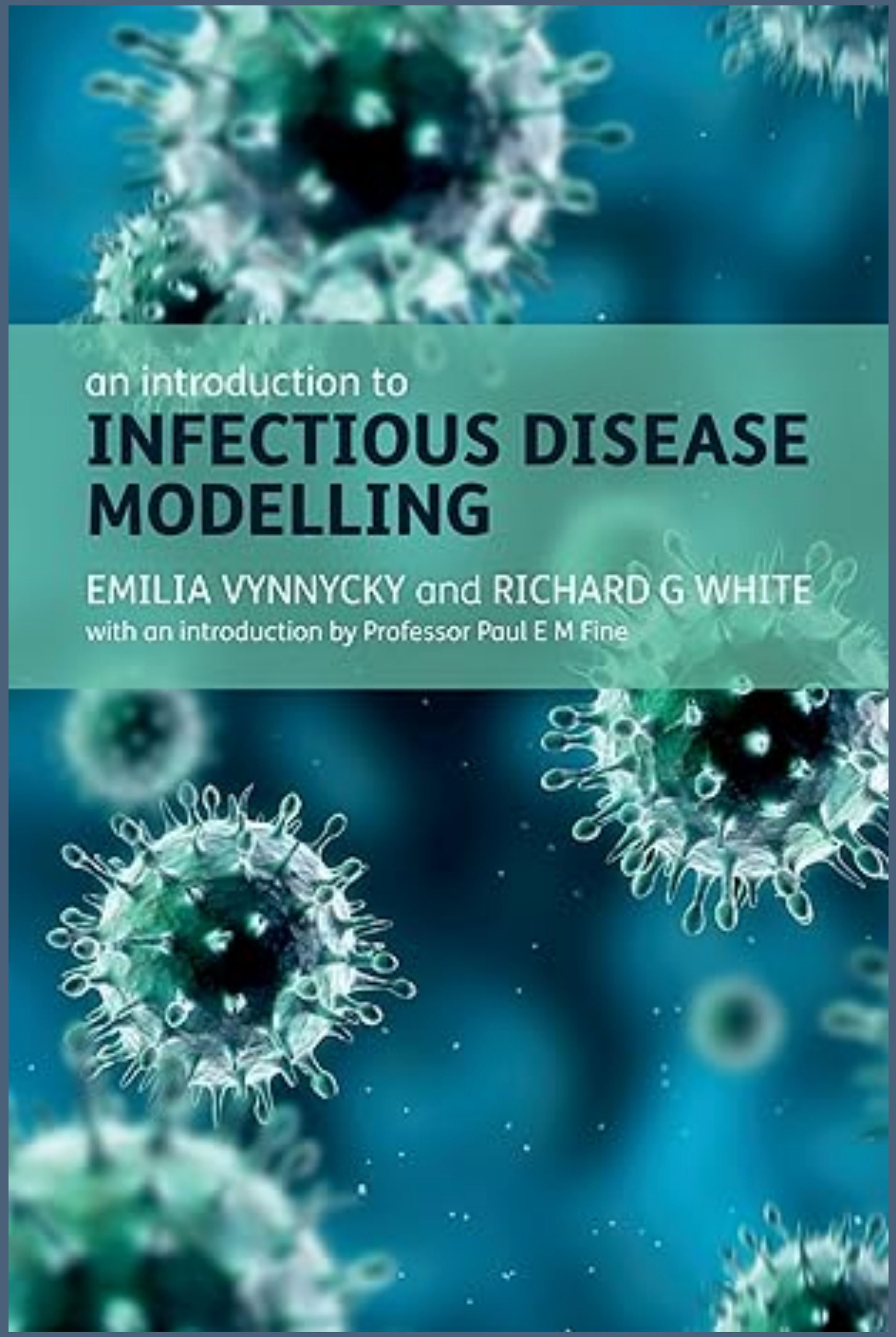
- Dr Adam Kucharski, mathematical epidemiologist at LSHTM and practising epidemiologist, including for COVID and Zika, and the BBC Pandemic project
- His popular science book on epidemic models was published on 13 February 2020
- One chapter covers the century-long history of the Susceptible-Infected-Recovered (SIR) epidemic models ads
- Epidemic models are also applied for other purposes including advertising messages, forecasting shootings in inner-city America



https://github.com/dpnolan/pandemic

Literature Survey - Maths Textbook

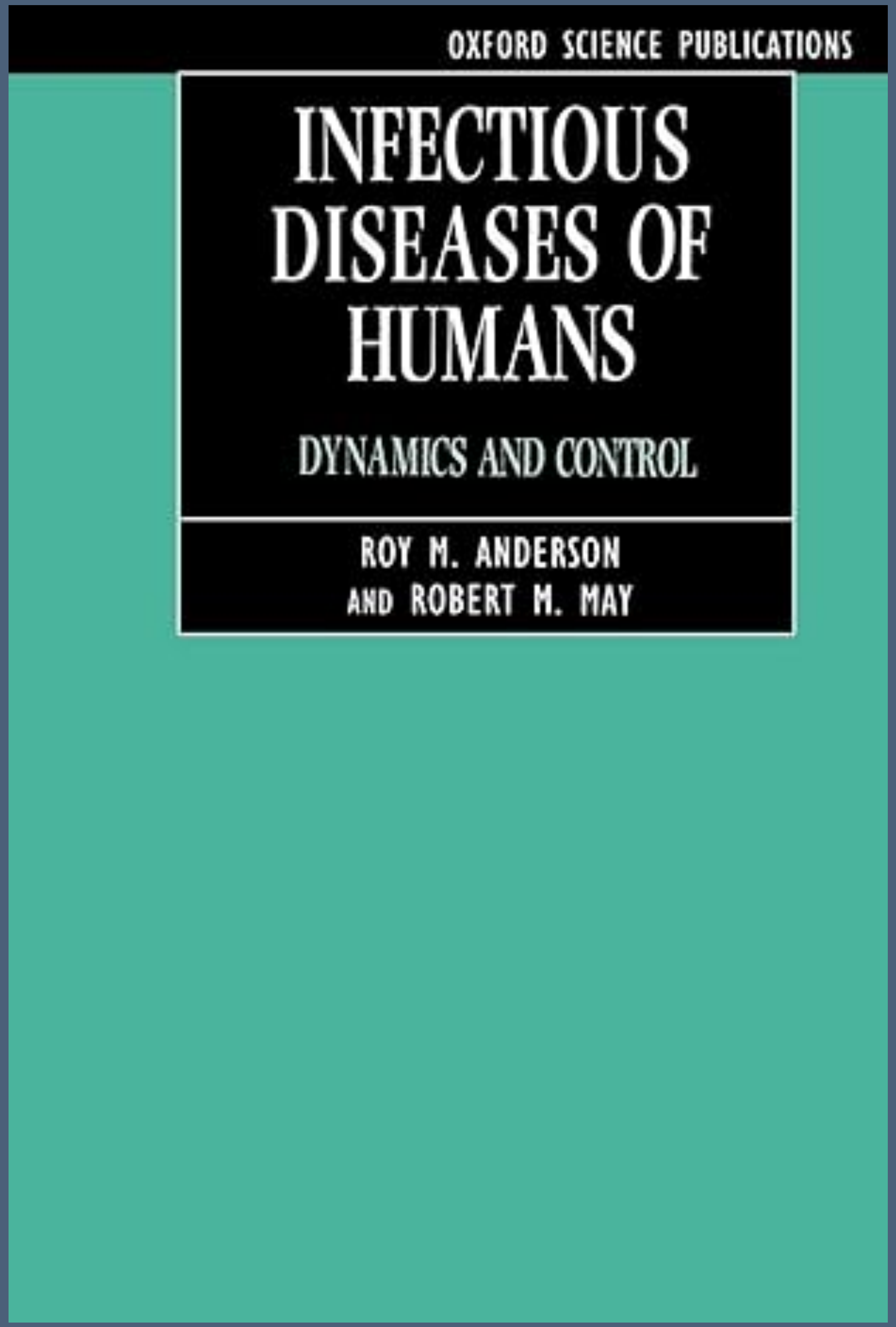
- Vynnycky and White give a broad survey of the mathematical models in this 2010 textbook
- A mathematical textbook in the treatment, no code
- Starts with Susceptible-Infected-Recovered(SIR) 'compartment' or 'population' models
- Difference and differential equations underly the SIR, first used more than 100 years ago to describe disease dynamics as a function of time by Kermack & McKendrick, two Scottish public health doctors working in India
- Simplest models include no reinfections, drift in virus, vaccinations, travel, age or behavioural effects
- These work well to model the first stage of the epidemic
- Agent, geographic, non-homogenous models are more complex



https://github.com/dpnolan/pandemic

Literature Survey - Model Dynamics

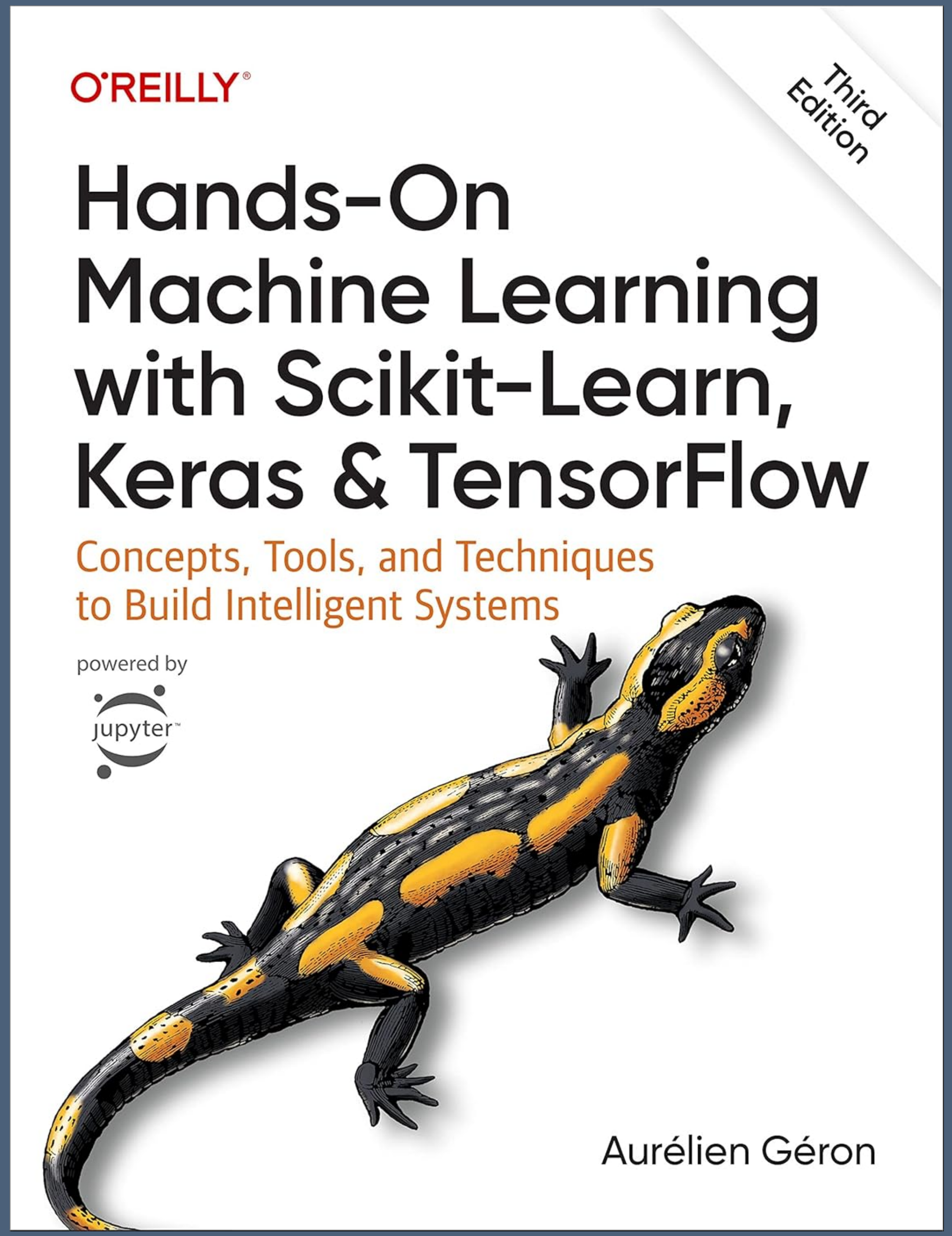
- Dynamics can be more complex: Work by Robert May et al. (1994) showed how chaotic, unpredictable behaviour arose within simple models without many inputs
- Real-world experience, as May worked as UK Chief Scientific Advisor, including on Foot and Mouth epidemic and bird 'flu



https://github.com/dpnolan/pandemic

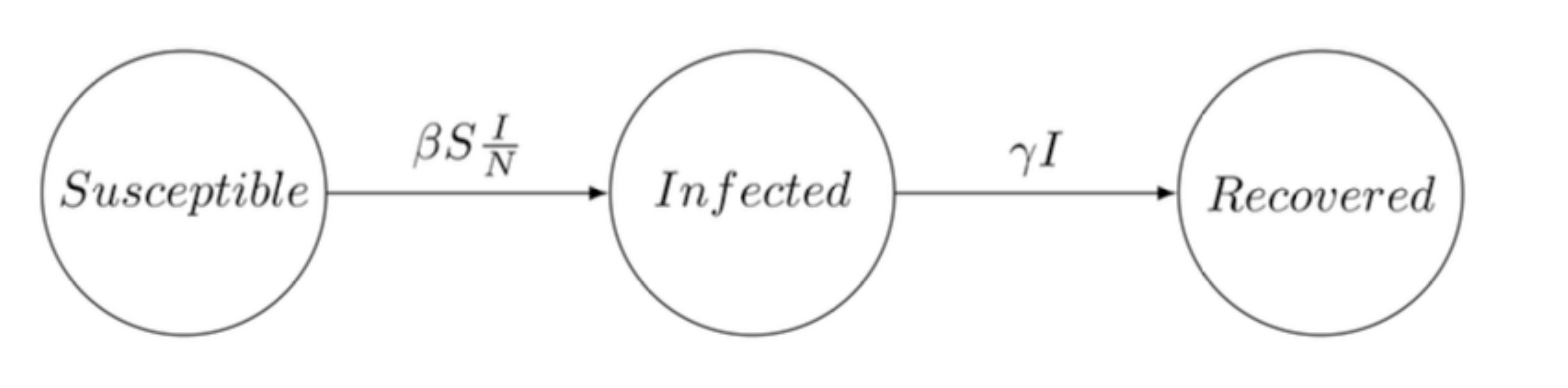
Literature Review - Machine Learning

- Should we use epidemic models or is an theoretical machine learning approach better?
- The Irish Epidemiological Modelling Advisory Group (IEMAG) was an independent team building models for the National Public Health Emergency Team (NPHET).
- Politicians gathered other data and models to challenge or 'peer review' the IEMAG's work. FF health minister Stephen Donnelly wanted more of a 'big data' and 'artificial intelligence' approach
- External consulting firms supported this under the "1 Government Centre" programme. They wanted to access many forms of data including card payments, CCTV and mobile phone network mobility data.
- IEMAG and NPHET did not back down and the alternative model results supported its scenarios



https://github.com/dpnolan/pandemic

The SIR Model - Specification

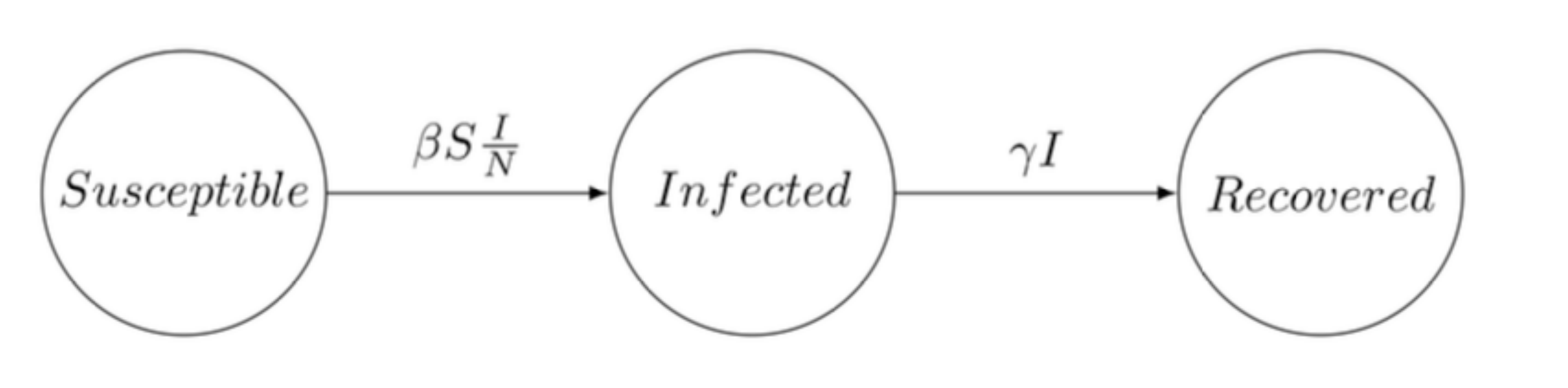


$$\frac{dS}{dt} = -\beta S \frac{I}{N}$$
$$\frac{dI}{dt} = \beta S \frac{I}{N} - \gamma I$$
$$\frac{dR}{dt} = \gamma I$$

- These divide the population into homogenous groups or compartments representing stages in the infection process
- SIR is commonly used for 'flu, stages are different for each disease e.g. SEIR, SIS, etc.
- Use difference and differential equations, math tools were available 100 years ago
- **N** is total population modelled, for example the population of the Republic of Ireland
- **S_t**, Susceptible may be infected, a portion become so based on beta, $\boldsymbol{\beta}$, the transmission rate, the constant rate of infectious contact between individuals
- **I_t**, Infected have caught the disease and show symptoms and are infectious
- **R_t**, Recovered, have got better or die at the recovery rate gamma, $\boldsymbol{\gamma}$, from among the Infected I_t.

https://github.com/dpnolan/pandemic

The SIR Ordinary Differential Equations



$$
\begin{aligned}
\frac{dS}{dt} &= -\beta S \frac{I}{N} \\
\frac{dI}{dt} &= \beta S \frac{I}{N} - \gamma I \\
\frac{dR}{dt} &= \gamma I
\end{aligned}
$$

- **S** reduced by new infections, calculated by infected as proportion of Beta, ***β***, the transmission rate, the constant rate of infectious contact between individuals, scaled by S(t) and I(t)
- **I_t**, Infected increase with new infections, transmit the infection in turn, reduced by recovery
- **R_t**, Recovered, have got better or die at a rate gamma, ***γ***, among the Infected
- **N**, the total population at start time, and N = S + I + R
- Personal characteristics - age, gender, recent travel, location, usually dealt with by separate models, or hierarchical model, but can be included as a model variable

https://github.com/dpnolan/pandemic

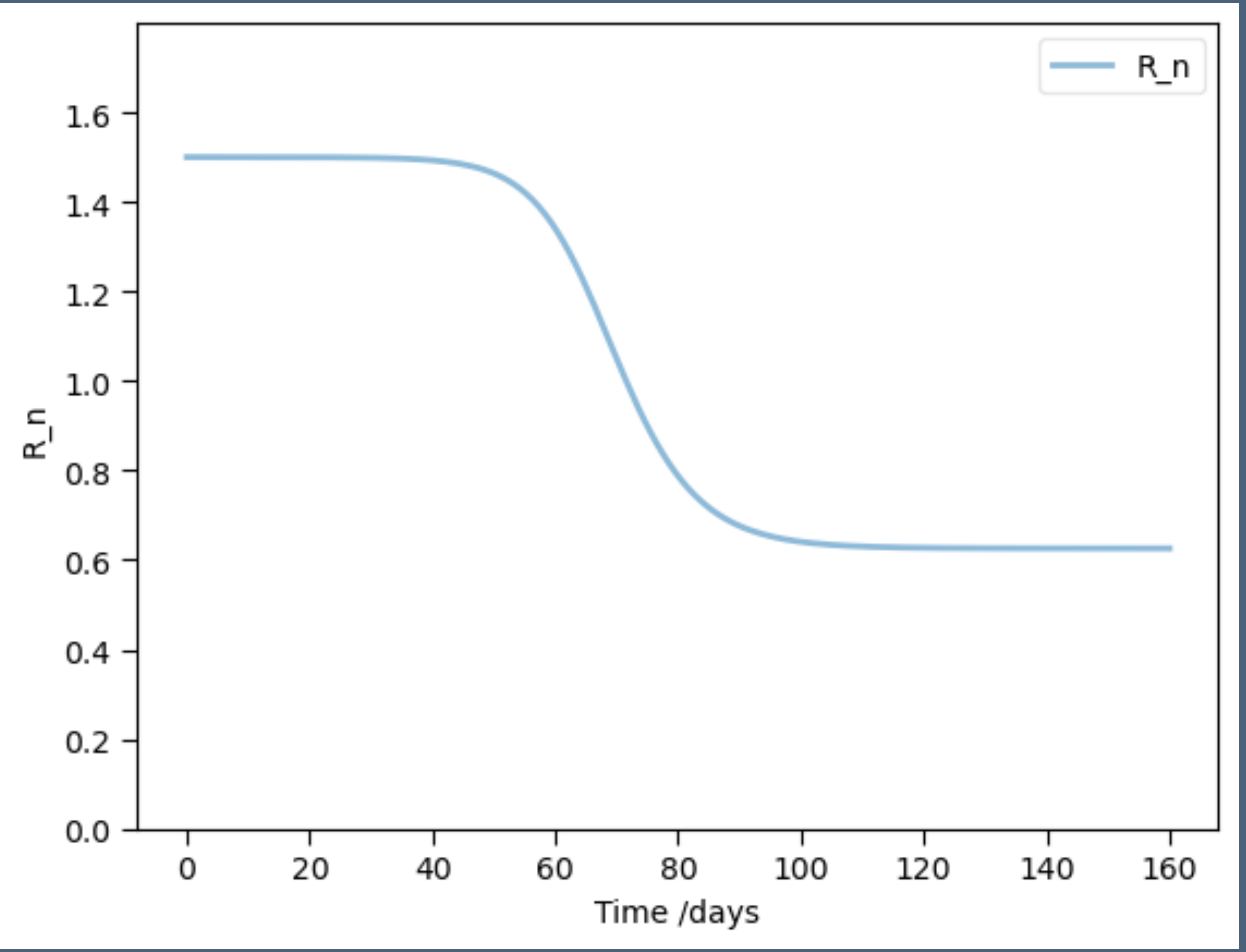
A Model to Answer the Research Question

- What is the past and present and possible future state of the epidemic?
- Inputs are the known observed infected numbers, I(t), observed by testing or symptoms
- Candidate models will start with the simple SIR
- Task is estimation, output for decision-makers are the SIR parameters
- Estimation done with Bayesian methods

https://github.com/dpnolan/pandemic

SIR Model Dynamics

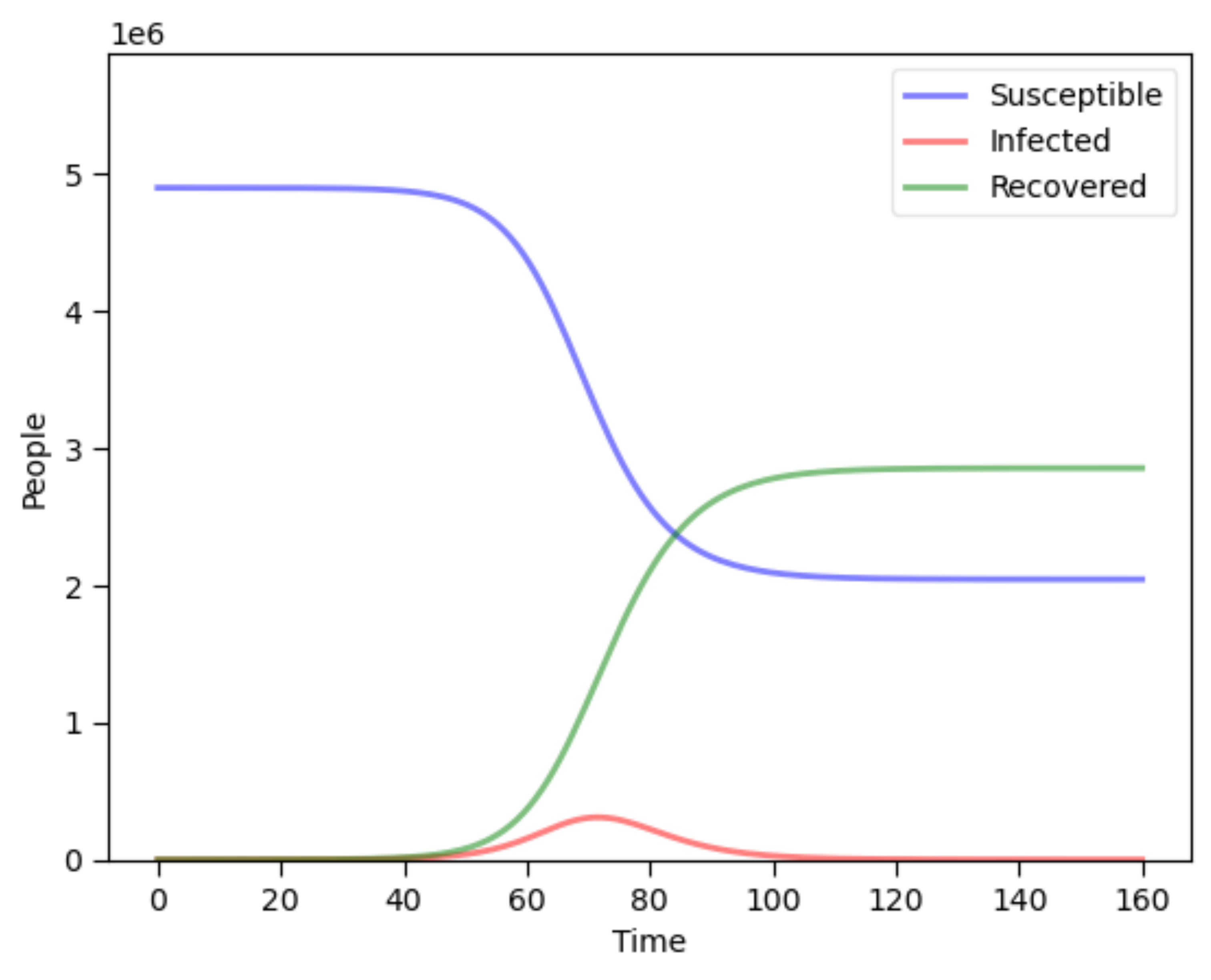
- PYMC directory, Jupyter Notebook 2
- **R_0** ('R zero') is the average number of infections from the first infected person in the population
- **R_0** = β/γ = 0.5/(1/3) * (S/N) = 1.5
- COVID began around 4. Seasonal 'flu typically is 1.5 to 2, measles around 20
- **R_t** or 'R number' is the variable used in forecasting future cases
- IF R_t <1, then the infection is not growing, if R_t>1, it will grow
- Over time, R_t declines as more of the population pass through infection, as S declines
- In our example, when S/N = 2/3 then, R_t=1, called the Herd Immunity Threshold. The epidemic will no longer grow, which vaccines or reinfections. Vaccination strategy targets this.
- 'Contagion' movie explainer: https://www.youtube.com/watch?v=X-YR3UlH3aA

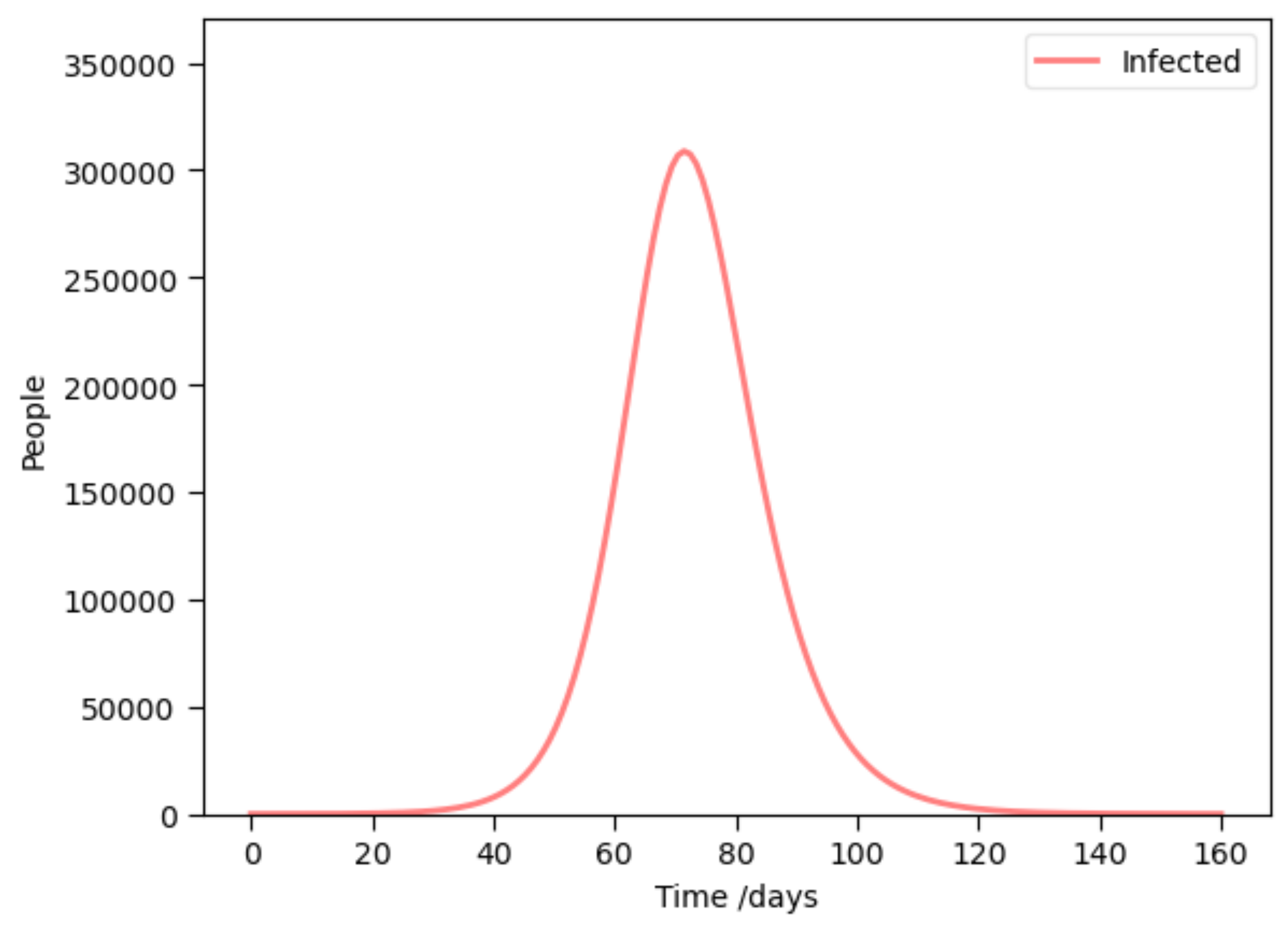


https://github.com/dpnolan/pandemic

SIR Model Dynamics

- Dynamics - typical changes over time, growth, rise and fall in infections
- See Jupyter workbook 1
- Flu example based on 'flu in Ireland, β = 0.5, **γ** = 1/3, N = 4.9 million people
- Susceptible become Infected, the Infected Recover or die
- Infections peak, but as a small part of the total population at any one time





https://github.com/dpnolan/pandemic

SIR Model Estimation

- Textbooks such as Vynnycky & White (2010) and a shelf full of others in Trinity's library do not cover estimation, to my surprise
- Searching during 2020 for an estimation of a simple population model, I found almost nothing published
- One simple approach was to solve the Ordinary Differential Equation (ODE) system
- One non-specialist, some Russian developers at Microsoft and students, implemented this in Python using scipy.integrate.odeint to model and scipy.optimize to fit the data up to June 2020: See Petrova et al. at https://github.com/shwars/SlidingSIR/tree/master

Bayesian Estimation - How and Why?

- Model the parameters as random variables to be estimated
- From the perspective of early to mid 2020, the underlying parameters for COVID were only going to become clear over time as evidence was gathered and published
- Making assumptions about distributions would be premature
- Some informative data is available early on and we best incorporate it into our analysis
- Expert opinion should be incorporated where useful
- Also, our uncertainty and updates to it as more and data is observed should be updated over time
- All these argue for a Bayesian rather than a simple ODE approach

Case Studies - GitHub 'papers' directory

- Grinsztajn et al. (2021) work in mathematical epidemiology and published a model estimating population models for COVID, working through from simple SIR, this is the closes I
    - Paper at https://doi.org/10.1002/sim.9164
    - Code + data https://github.com/charlesm93/disease_transmission_workflow/tree/main
    - also in the STAN documentation https://mc-stan.org/learn-stan/case-studies/boarding_school_case_study.html
- A small, simple dataset on a 1978 'flu outbreak in an English school is commonly used in this literature, including by Grinsztajin
- Paper https://pmc.ncbi.nlm.nih.gov/articles/PMC1603269/pdf/brmedj00115-0064.pdf
- Data https://www.kaggle.com/datasets/antonymgitau/influenza-england-1978-school-csv
- IEMAG published a second edition of their model methodology in October 2020, updating an earlier model from May 2020.
    - Gleeson JP, Murphy TB, O'Brien J, O'Sullivan D. (2021) "A population-level SEIR model for COVID-19 scenarios (updated)," https://assets.gov.ie/122667/8379f0cc-5be3-4c89-9a1e-3b7328ae03af.pdf
    - Code and data https://github.com/obrienjoey/ireland_covid_modelling
    - Full CIDR dataset 19.geohive.ie/datasets/d8eb52d56273413b84b0187a4e9117be_0/about
- A wider historical literature review is in Lit Review Epidemic Models.pdf in the GitHub

Bayesian Estimation

- Count discrete, non-independent events, which occur in clusters in time or space
- The first version of the IEMAG model in May 2020 used the Poisson distribution.
- Exponential distribution is often used model time between outbreaks, where **λ** parameter is the time between events and also the mean of the Poisson distribution of events per year.
- Negative Binomial distribution was used instead to count the cases: was used instead in the data and model from November 2020, probably to deal with greater variation in the pandemic case numbers than the Poisson, which assumes mean and variance are the same value.
- Inverse overdispersion with exponential distribution with mean 1/**γ**.
- NB uses I(t) as the mean and a parameter for inverse overdispersion phi, **ϕ** i.e. smaller phi means more dispersion
- **β** and **γ** are assumed non-negative and different priors were used in repeated estimates.

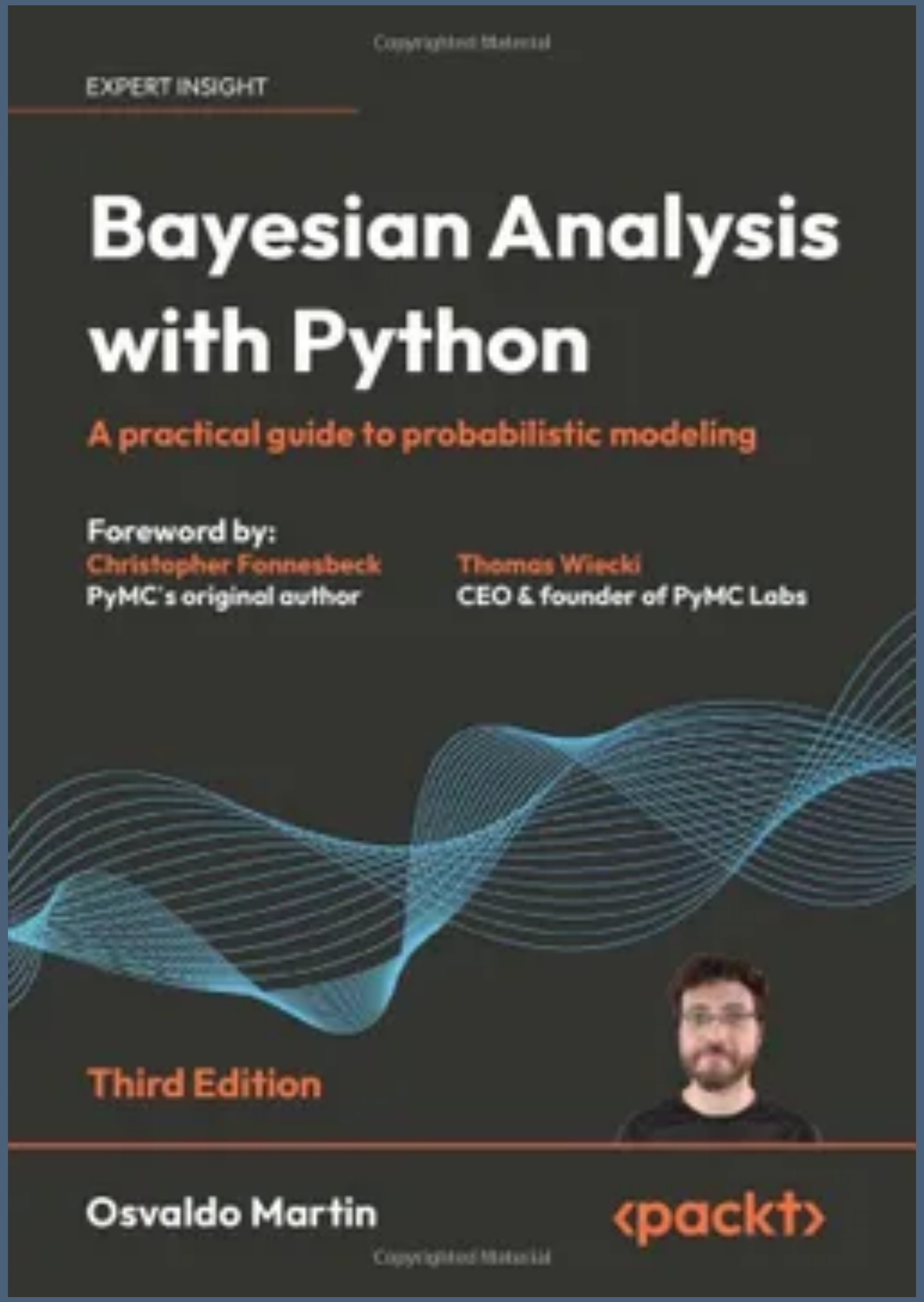
$$\frac{dS}{dt} = -\beta S \frac{I}{N}$$
$$\frac{dI}{dt} = \beta S \frac{I}{N} - \gamma I$$
$$\frac{dR}{dt} = \gamma I$$

https://github.com/dpnolan/pandemic

Workflow

- Martin (2024) and PyMC core developer Thomas Wiecki suggest something like https://gist.github.com/twiecki/43def0aa16ee6a23f822a124eb429958

1. Plot the data
2. Build model
3. Check prior predictions
4. Fit models
5. Assess convergence
6. Run posterior predictive check
7. Improve model



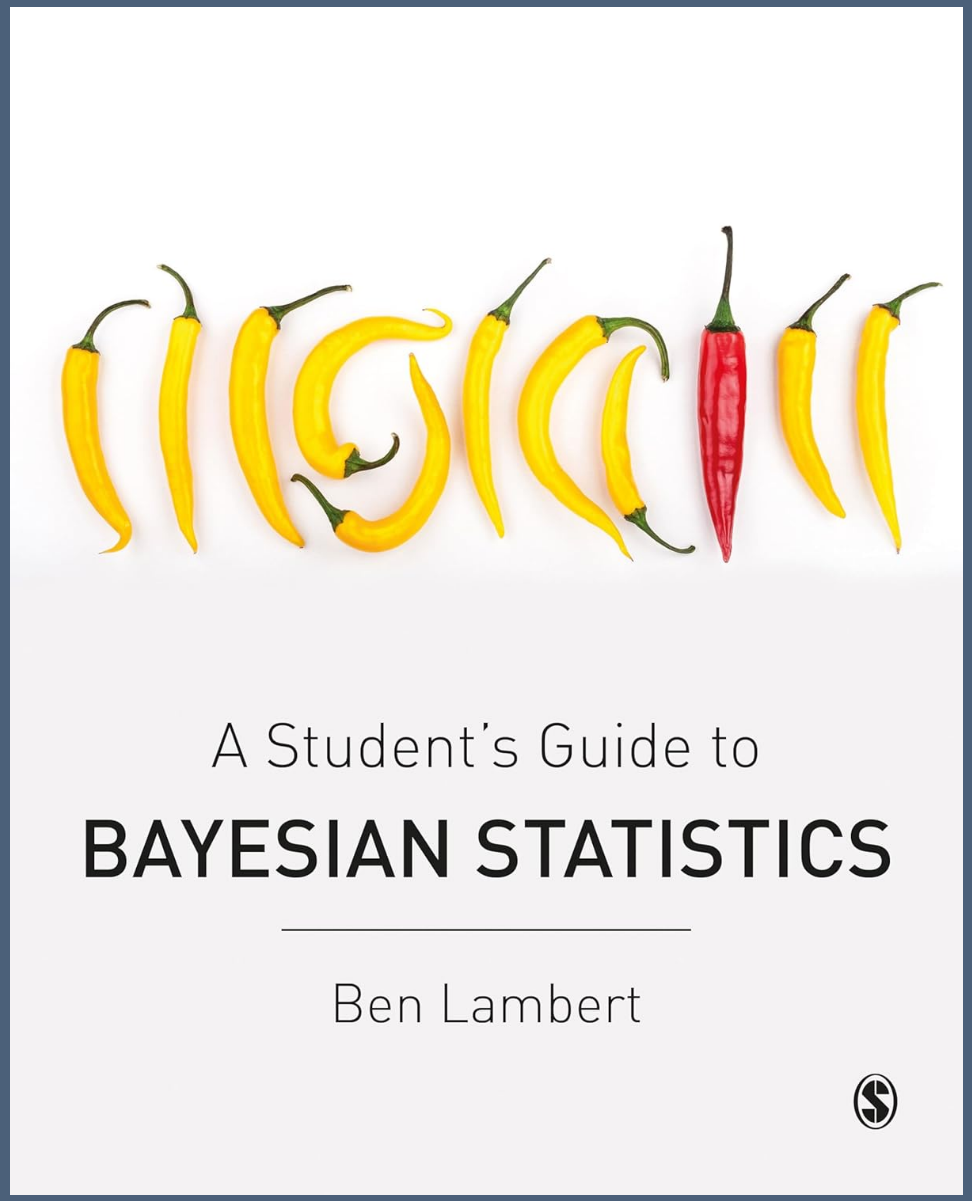
https://github.com/dpnolan/pandemic

Bayes and Markov Chain Monte Carlo

- Without a simple arithmetic Conjugate Prior from our chosen distributions, we estimate using simulation with MCMC
- Monte Carlo - simulate by random draws
- Markov Chain - future state depends on the current state only
- We sample from P(**θ** | Data), distribution of Probability of the model+parameters using the prior and likelihood distributions to set the frequency with which we sample from each region of the posterior
- From this we calculate the characteristics of our posterior distribution
- PyMC uses No U-Turns (NUTS) version of the Hamiltonian sampling algorithm, searching parameter space using gradient, then stopping search when the sample search turns back to its previous steps on the path
- STAN uses Hamiltonian Monte Carlo, with the NUTS engine

$$P(\theta \mid D) = \frac{P(D \mid \theta) \cdot P(\theta)}{P(D)}$$

$$P(\theta \mid D) \propto P(D \mid \theta) \cdot P(\theta)$$



https://github.com/dpnolan/pandemic

Python Libraries

- Usual data analysis libraries
- **SciPy** for the old Fortran **odeint**() solver
- **PyMC** for running the MCMC, ran on Windows 11
- **STAN** language was used with **cmdstanpy** a wrapper for STAN, in Windows System Linux, and on Google Colab.
- Installing **PyStan**, **cmdstanpy** failed repeatedly in Windows and MacOS
- **ArviZ** was for MCMC diagnostics and metrics
- No one platform would run both, hence the separate files and static versions in the GitHub
- Runs on Google Colab and with one 4090 GPU , runs took 5-25 minutes

https://github.com/dpnolan/pandemic

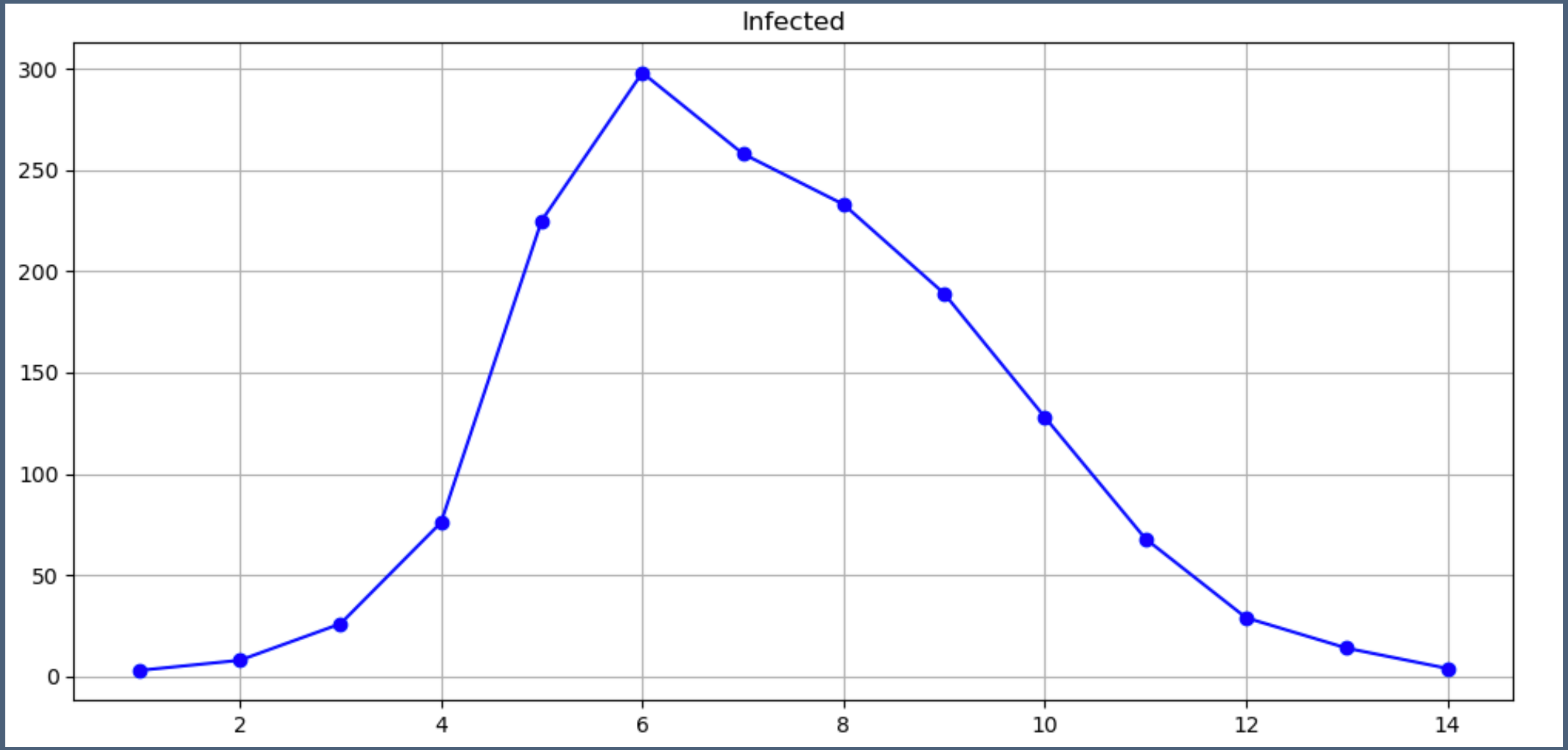
Diagnostics from ArviZ

- Require log-likelihood from PyMC or STAN output in **idata** output
- Summary statistics for estimated parameters
- **R-hat** - compares between-chain variance to within-chain variance for the parameter: if chains have mixed well and are sampling the same stationary distribution, those variances match and R-hat <= 1.05
- **Trace plots** show marginal distribution of estimated parameter and time-series per chain of samples - look for convergence and the shape of the estimates
- **Pair plots** - joint posterior of selected parameters often with divergent transitions overlaid. They show correlations or non-linear relationships,or multimodality
- **Energy plots** shows the Hamiltonian used by HMC/NUTS, movement, wide range means exhaustive search
- **Posterior plot** - show the distribution, the mean, mode and dispersion and compare to the original sample
- **WAIC** - Watanabe Information Criterion, estimates out-of-sample predictive accuracy from MCMC draws using the pointwise log-likelihood, this incorporates the posterior uncertainty in assessing accuracy, the higher the better
- **LOO -** Leave One Out, another Information Criterion, higher the better

https://github.com/dpnolan/pandemic

School Dataset

- 'Flu outbreak in an English school, dataset published in 1978
- PyMC - See Jupyter Notebook 3
- STAN - Seee STAN/ CmdStanPy1.ipyn section 'School Example'



School Case-Study - SIR ODE

- SIR function is defined in Python
- I found the logic in a comment on a PyMC forum, the only published SIR I found
- Marginalisation - we ignore R(t), integrating over all values

Define the model

si_func describes the ODE for the SIR model Marginalised estimate, ignoring R(t), which is not available, and calculating S(t) as N minus I(t) This function is based on: https://github.com/Priesemann-Group/covid19_inference/blob/master/covid19_inference/model/compartmental_models/SIR.py

In [12]:

```
# SIR function estimating S, I and R
# This function is based on: \
# https://github.com/Priesemann-Group/covid19_inference/blob/master/covid19_inferenc

def sir_func(S_t, I_t, new_I0, R_t, beta, gamma, N): #add R
#def sir_func(S_t, I_t, new_I0, beta, gamma, N):
    new_I_t = beta * S_t * ( I_t / N )
    new_R_t = gamma * I_t
    S_t = S_t - new_I_t
    I_t = I_t + new_I_t - new_R_t
    R_t = R_t + new_R_t # add R_t

    #I_t = pt.clip(I_t, -1, N)  # for stability
    I_t = pt.clip(I_t, 0, N)  # for stability
    S_t = pt.clip(S_t, 0, N)
    R_t = pt.clip(R_t, 0, N) #add R_t

    #return S_t, I_t, new_I_t
    return S_t, I_t, new_I_t, R_t # add R
```

School Case-Study - Define the Priors

pandemic / PYMC_notebooks / school.ipynb

Blame 1411 lines (1411 loc) · 745 KB

- **pt.zeros** Variables, S, I and R are initialised as tensors with zero values for the model variables
- **pt.subtensor.inc_subtensor** PyTensor expression used to initialize the values of the first element of tensors S and I (for Susceptible and Infected)
- **scan** Like in TensorFlow, we have initiated a graph, this then iterates through the ODE as defined in **si_func**

In [14]:

```
with pm.Model() as mod:

    # Priors
    beta = pm.HalfNormal('beta',1) # match Semenova et al. priors
    #beta = pm.Wald('beta', 2, 1)

    gamma = pm.HalfNormal('gamma', 1) # match Semenova et al. priors
    #gamma = pm.Wald('gamma', 1, 1)

        # Variables
    S = pt.zeros(n_days)
    I = pt.zeros(n_days)
    R = pt.zeros(n_days)

    S = pt.subtensor.inc_subtensor(S[0], N - infected[0])
    I = pt.subtensor.inc_subtensor(I[0], infected[0])

    #outputs, progress = scan(
    outputs, _ = scan(
        fn=si_func,
        outputs_info=[S_begin, I_begin, new_I0],
        non_sequences=[beta, gamma, N],
        n_steps=n_days
    )
    S_t, I_t, new_I_t = outputs
```

School Case-Study - MCMC Functions, Sampling, Logs, Output to ArviZ

```python
    # Likelihood
    phi_inv = pm.Exponential("phi_inv", 0.1)
    #phi_inv = pm.Exponential("phi_inv", 5)

    y = pm.NegativeBinomial("y", mu=I_t, alpha=phi_inv, observed=infected)

    R0 = pm.Deterministic("R0", beta/gamma)
    recovery_time = pm.Deterministic("recovery time", 1/gamma)

    S_t = pm.Deterministic("S_t", S_t)
    I_t = pm.Deterministic("I_t", I_t)

    # with open("sir_log.csv", mode="a", newline="") as file:
    #     writer = csv.writer(file)
    #     writer.writerow([S_t, I_t, new_I_t])

with mod:

    idata_log = pm.sample(1000, idata_kwargs={"log_likelihood": True}, chains=4)
    idata_log.extend(pm.sample_posterior_predictive(idata_log))

summ = az.summary(idata_log, hdi_prob=0.9)
```

School Case-Study - STAN code

- Line 60 - Priors defined
- Line 74 - Negative binomial for infections
- Line 76 - log-likelihood generated for ArviZ to access

main pandemic / STAN / STAN / **sir_negbin2_log2.stan**

Code Blame 81 lines (71 loc) · 1.79 KB

```
51      {
52        array[2] real theta;
53        theta[1] = beta;
54        theta[2] = gamma;
55
56        y = integrate_ode_rk45(sir, y0, t0, ts, theta, x_r, x_i);
57      }
58    }
59    model {
60      //priors
61      beta ~ normal(2, 1);
62      gamma ~ normal(0.4, 0.5);
63      phi_inv ~ exponential(5);
64
65      //sampling distribution
66      cases ~ neg_binomial_2(col(to_matrix(y), 2), phi);
67    }
68    generated quantities {
69      real R0 = beta / gamma;
70      real recovery_time = 1 / gamma;
71      array[n_days] real pred_cases;
72
73      //col(matrix x, int n) - The n-th column of matrix x. Here the number of infected people
74      pred_cases = neg_binomial_2_rng(col(to_matrix(y), 2), phi);
75
76      vector[n_days - 1] log_lik; // Declare a vector for log-likelihood values
77      for (i in 1:(n_days - 1)) {
78        // Calculate the log-likelihood for each observation using the infected number as the mean
79        log_lik[i] = neg_binomial_2_lpmf(cases[i] | col(to_matrix(y), 2)[i], phi);
80     }
81    }
```

School Case-Study - Calling STAN

- Feed STAN code and data to the simulation via CmdStanModell()

pandemic / STAN / CmdStanPy1_1.ipynb

Go to file

dstanpy notebook

b704ce9

In [ ]:
```
# --- Time axis used by Stan ODE block ---
t0 = 0.0
# Typical choice: one time unit per row. Stan expects strictly increasing ts:
ts = np.arange(1, n_days + 1, dtype=float)
```

In [ ]:
```
# --- Population size (N) and initial state y0 ---
N = 763

# Initial conditions [S0, I0, R0]; common choice starts with 1 infected, 0 recovered.
y0 = np.array([N - 1, 1, 0], dtype=float)
```

In [ ]:
```
# --- Build Stan data list (must match the Stan file's data block) ---
stan_data = {
    "n_days": int(n_days),
    "y0": y0.astype(float),
    "t0": float(t0),
    "ts": ts.astype(float),
    "N": int(N),
    "cases": cases.astype(int),
}
```

In [ ]:
```
model2 = CmdStanModel(stan_file=os.path.join(BASE,'stan','sir_negbin2_log2.stan'), force_compile=True,
                      compile="stan --build=make TBB=true")
fit2 = model2.sample(data=stan_data, seed=123, adapt_delta=0.8)
```

School Case-Study - Summary

- Stats look close to those in the published model
- R-hat shows good matching of variance in search within and across chains, a sign of a robust and wide search

In [15]:

```
az.summary(idata_log, var_names=["beta", "gamma", "R0", "recovery time"],hdi_prob=0.9)
```

Out[15]:

	mean	sd	hdi_5%	hdi_95%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
beta	1.735	0.058	1.639	1.816	0.001	0.001	1932.0	1362.0	1.0
gamma	0.477	0.035	0.416	0.529	0.001	0.001	2162.0	2070.0	1.0
R0	3.660	0.349	3.103	4.210	0.009	0.006	1813.0	1577.0	1.0
recovery time	2.107	0.152	1.852	2.348	0.003	0.002	2162.0	2070.0	1.0

By comparison, the estimates from Simonova et al.2021 were beta 1.73 and gamma 0.54

School Case-Study - Diagnostics WAIC and LOO

In [24]:
```
waic_log = az.waic(idata_log)
waic_log
```

```
C:\ProgramData\anaconda3\envs\pymc_env_1\Lib\site-packages\arviz\stats\stats.py:1653: UserWarning: For one or more
samples the posterior variance of the log predictive densities exceeds 0.4. This could be indication of WAIC start
ing to fail.
See http://arxiv.org/abs/1507.04544 for details
  warnings.warn(
```

Out[24]:
```
Computed from 4000 posterior samples and 14 observations log-likelihood matrix.

          Estimate       SE
elpd_waic   -61.96     5.08
p_waic        2.33        -

There has been a warning during the calculation. Please check the results.
```

In [25]:
```
loo_log = az.loo(idata_log)
loo_log
```

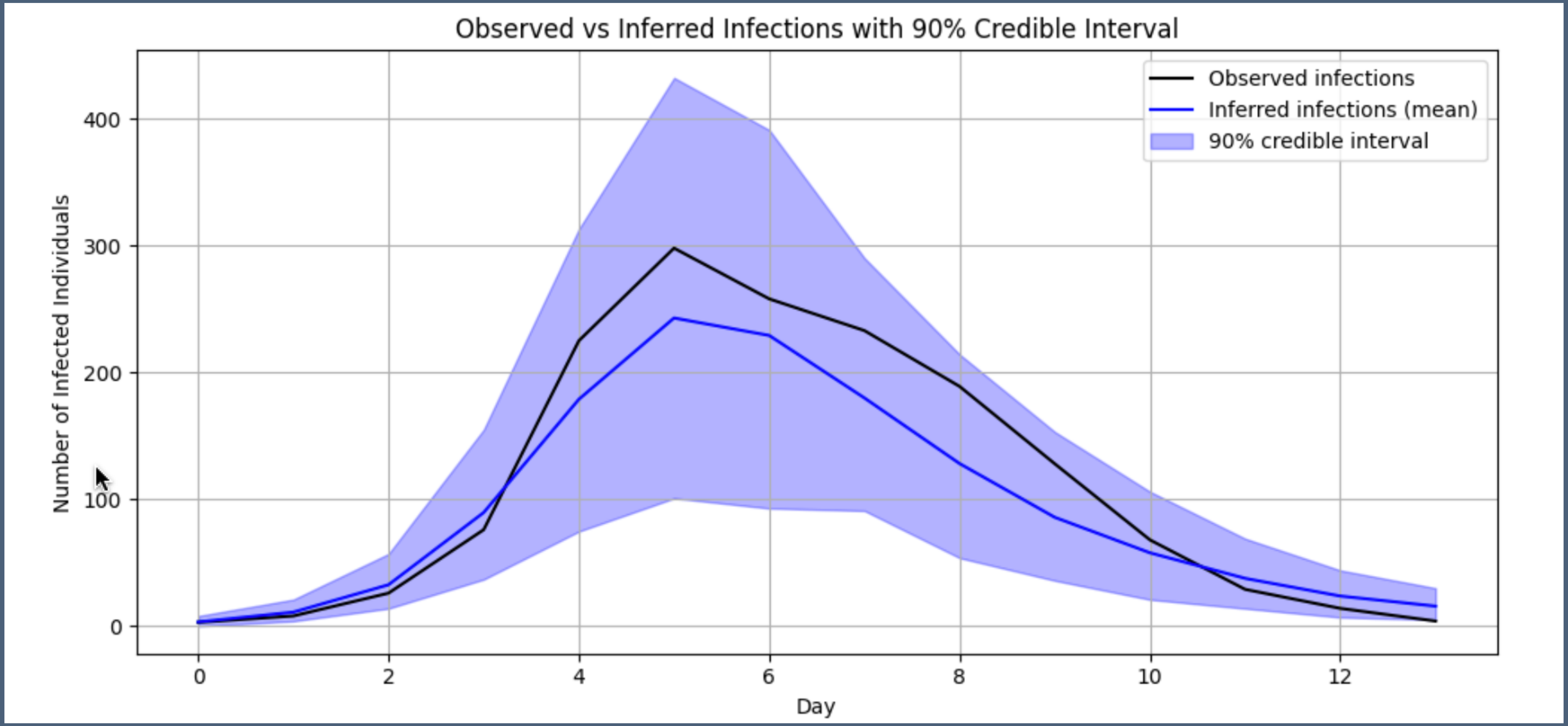
Out[25]:
```
Computed from 4000 posterior samples and 14 observations log-likelihood matrix.

         Estimate       SE
elpd_loo   -62.08     5.12
p_loo        2.45        -
------

Pareto k diagnostic values:
                         Count   Pct.
(-Inf, 0.70]   (good)       14  100.0%
   (0.70, 1]   (bad)         0    0.0%
   (1, Inf)   (very bad)    0    0.0%
```

School Case-Study - Diagnostics - Graph

- Diagnostics - posterior v observations

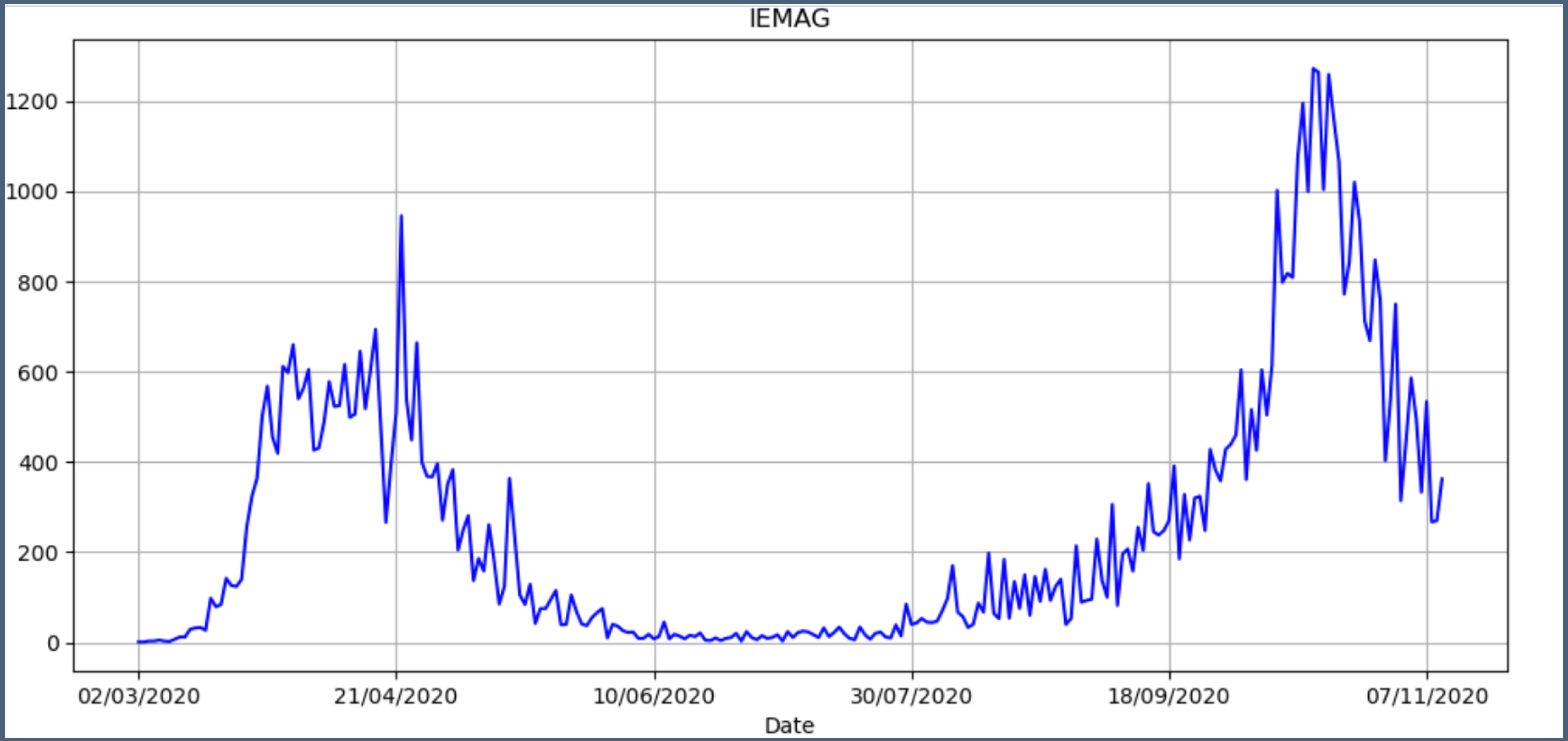
- Most commonly reported

- This is from the STAN / ArviZ output



IEMAG Data

- See Jupyter Notebook 4
- STAN - Seee STAN/ CmdStanPy1.ipyn section 'IEMAG Irish Pandemic Data'
- IEMAG models fitted in May and October
- Strong seasonality and effect from control measures



IEMAG Data Run 1 - Summary Statistics

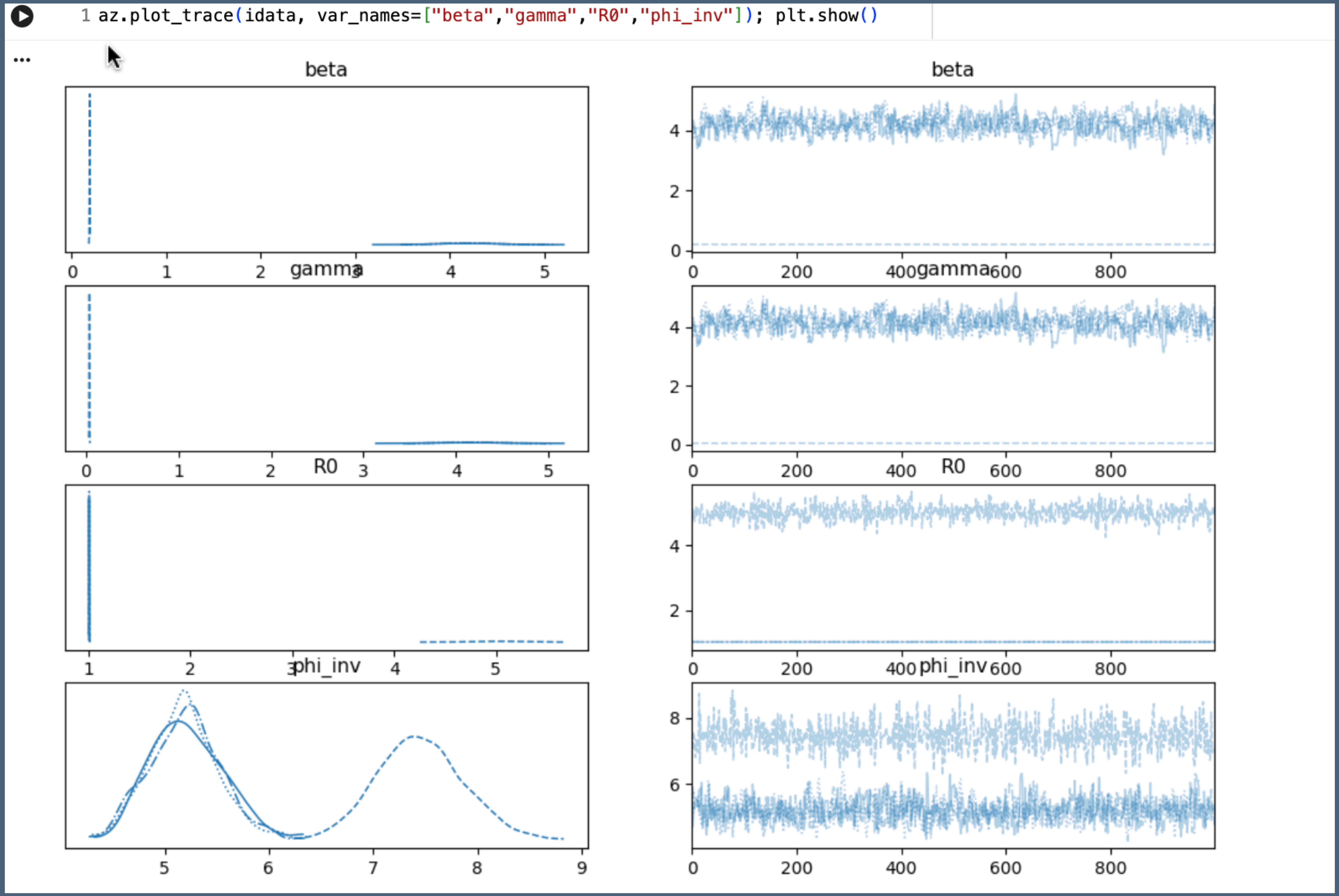
- First model sir_incidence3.stan
- Use same priors as Swiss data in Simonova et al.
- beta ~ normal(2, 1),
- gamma ~ normal(0.4, 0.5);
- phi_inv ~ exponential(5);
- Estimates show very high r_hat, chains don't converge
- WAIC -1967

```
idata = az.from_cmdstanpy(posterior=fit4)

# Summary incl. mean, sd, mcse, r_hat, ess_bulk/tail
print(az.summary(idata, var_names=["beta","gamma","R0","phi_inv"], round_to=3))
```

```
          mean     sd  hdi_3%  hdi_97%  mcse_mean  mcse_sd  ess_bulk  \
beta     3.198  1.756   0.183    4.648      0.866    0.489     7.234
gamma    3.126  1.802   0.033    4.605      0.889    0.502     7.213
R0       2.012  1.738   1.008    5.170      0.864    0.501     7.213
phi_inv  5.763  1.043   4.527    7.799      0.489    0.272     7.249

         ess_tail  r_hat
beta       32.220  1.533
gamma      32.073  1.533
R0         30.238  1.533
phi_inv    32.795  1.535
```

IEMAG Data Run 1 - Summary Statistics

- Trace diagrams show no distribution of estimates
- Split chain 4 shows data may be multimodal non-stationary



IEMAG Data Run 2 - Irish Data with Tighter Priors

- Second model sir_incidence4.stan
- Use tighter priors as in Simonova et al.
- beta ~ normal(2, 0.5);
- gamma ~ normal(0.4, 0.2);
- phi_inv ~ exponential(10);
- Estimates show very high r_hat, chains don't converge
- Still very bad
- WAIC -2007

```
1 print(fit5.summary())
2 fit5.summary().to_csv("fit5_summary.csv", index=True)
```

```
                      Mean       MCSE     StdDev            5%           50%  \
lp__         -2.059520e+03  10.001800  14.208800 -2.084840e+03 -2.051550e+03
gamma         1.608150e+00   0.641072   0.913977  3.670950e-02  2.071080e+00
beta          1.689920e+00   0.612127   0.873153  1.891770e-01  2.129230e+00
phi_inv       5.936120e+00   0.310434   0.545457  5.199480e+00  5.834250e+00
y[1,1]        4.900000e+06        NaN   0.000000  4.900000e+06  4.900000e+06
...                    ...        ...        ...           ...           ...
log_lik[249] -9.616300e+00   0.317632   0.490609 -1.056390e+01 -9.438710e+00
log_lik[250] -8.747090e+00   0.223026   0.342490 -9.405150e+00 -8.621940e+00
log_lik[251] -1.000270e+01   0.346031   0.547155 -1.105250e+01 -9.824080e+00
log_lik[252] -8.401920e+00   0.186706   0.289392 -8.954140e+00 -8.301690e+00
log_lik[253] -8.473940e+00   0.189883   0.297651 -9.037020e+00 -8.374600e+00

                       95%    N_Eff   N_Eff/s     R_hat
lp__         -2.050030e+03  2.01818  0.001865  12.21050
gamma         2.340670e+00  2.03263  0.001878   8.36178
beta          2.396290e+00  2.03469  0.001880   8.06087
phi_inv       6.965370e+00  3.08732  0.002853   1.64127
```

IEMAG Data Run 3 - Irish Data, first 3 months

- Original priors model sir_incidence3.stan
- Avoid periods of lockdowns and reinfections handled with extra parameters in Simonova et al.
- Estimates show high but not absurd r_hat
- Not great
- WAIC -58

```
idata = az.from_cmdstanpy(posterior=fit6)

# Summary incl. mean, sd, mcse, r_hat, ess_bulk/tail
print(az.summary(idata, var_names=["beta","gamma","R0","phi_inv"], round_to=3))
```

```
          mean     sd  hdi_3%  hdi_97%  mcse_mean  mcse_sd  ess_bulk  \
beta     2.332  0.470   1.527    3.147      0.074    0.109    56.276   
gamma    1.974  0.562   1.068    2.942      0.092    0.140    51.545   
R0       1.352  0.888   1.057    1.399      0.218    0.557    47.992   
phi_inv  2.216  0.328   1.615    2.836      0.046    0.014    40.672   

         ess_tail  r_hat  
beta       18.198  1.082  
gamma      18.451  1.081  
R0         18.195  1.079  
phi_inv   128.764  1.086  
```

IEMAG Data Run 4 - Irish Data, first 3 months, tighter priors

- Original priors model sir_incidence3.stan
- Avoid periods of lockdowns and reinfections handled with extra parameters in Simonova et al.
- Estimates show high but not absurd r_hat
- Worse again
- WAIC -428

```
idata = az.from_cmdstanpy(posterior=fit6)

# Summary incl. mean, sd, mcse, r_hat, ess_bulk/tail
print(az.summary(idata, var_names=["beta","gamma","R0","phi_inv"], round_to=3))
```

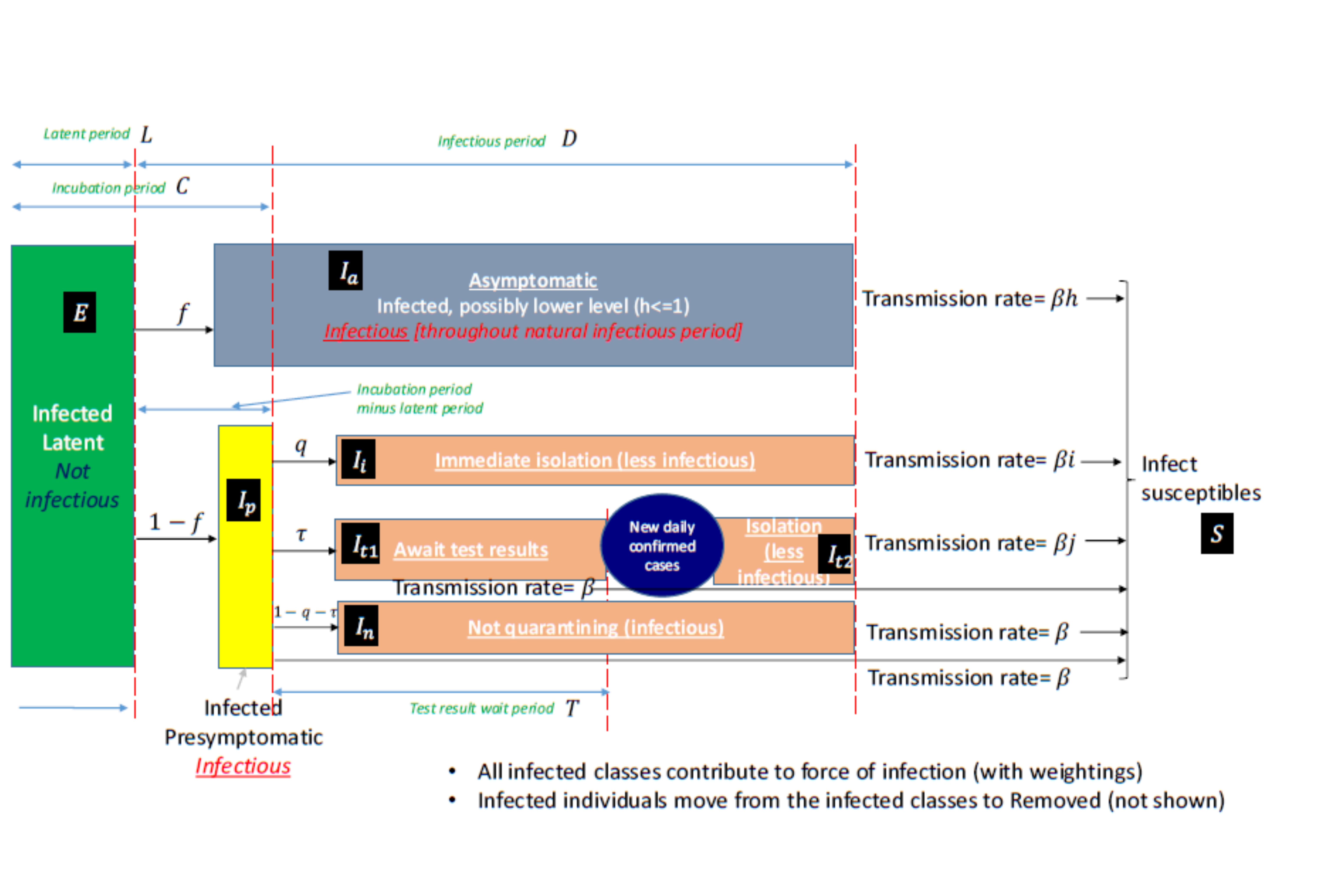
```
          mean     sd  hdi_3%  hdi_97%  mcse_mean  mcse_sd  ess_bulk  \
beta     2.332  0.470   1.527    3.147      0.074    0.109    56.276
gamma    1.974  0.562   1.068    2.942      0.092    0.140    51.545
R0       1.352  0.888   1.057    1.399      0.218    0.557    47.992
phi_inv  2.216  0.328   1.615    2.836      0.046    0.014    40.672

         ess_tail  r_hat
beta       18.198  1.082
gamma      18.451  1.081
R0         18.195  1.079
phi_inv   128.764  1.086
```

IEMAG Data Model Outcomes

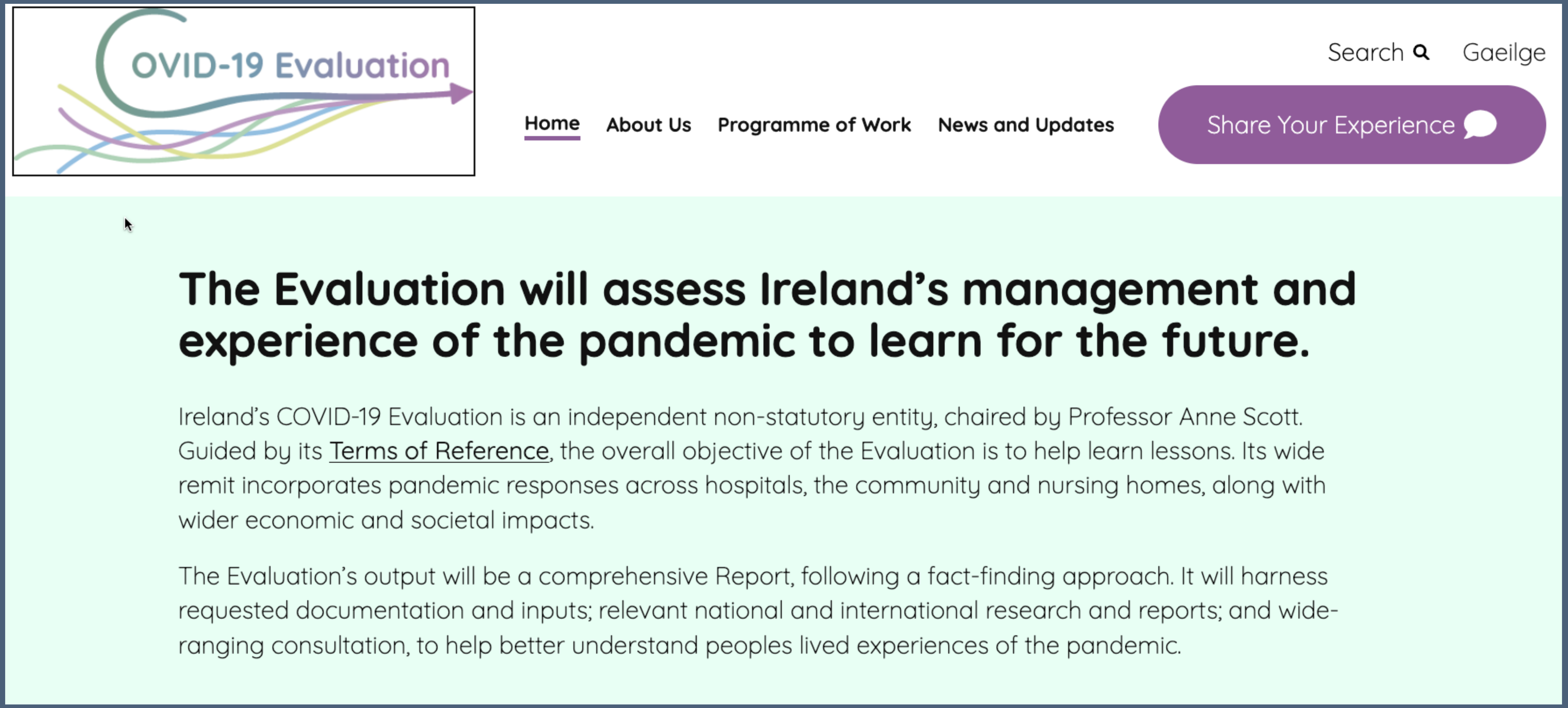
- The fit for the simple PYMC model is very poor
- This was not, as I thought first, a bug, but a misspecification of the model
- Tightening the prior conditions led to better performance on the sampling Rhat and WAIC
- Moving to estimation from Feb to Oct 2020 to within a narrower period of 3 months' data, initial spread, peak, then fall in summer and autumn, performed better this way also also, likely due to stationary in the data
- The fit is not perfect, but the process seems to be productive; more factors should improve performance

Next Steps

- PyMC and STAN
- Compare with Northern Ireland's similar epidemic model, perhaps in one hierarchical model
- Social network models
- Compare with UK models https://www.gov.uk/government/publications/reproduction-number-r-and-growth-rate-methodology/reproduction-number-r-and-growth-rate-methodology#fnref:1
- Implement age-based model
- Add economic effects



The Irish COVID Enquiry



- The Irish Epidemiological Modelling Advisory Group (IEMAG) team head Prof. Philip Nolan discusses his experience at an RIA panel, commenting that the 'follow the science' approach was authoritarian https://www.youtube.com/watch?v=Estl2VBSKcc

- Irish enquiry into the pandemic is now under way by a panel of academics scrutinising the plans and responses, but with relatively limited powers, no cross-examination and no power of compel appearance. Contact details are online at https://www.covid19evaluation.ie/

- My own view is that the analysis needs prior preparation, probably with nominated specialists and a menu of data sources, models and tools shared within the EU and globally, with the data infrastructure set up in advance

- https://github.com/dpnolan/pandemice

Conclusions

- PyMC can estimate simple SIR models, coming close to published results or simulations
- Ecosystem of libraries and published research for epidemiology in STAN is larger, better-supported and gives same output
- Setup for PyMC was harder than STAN
- SIR models are the transparent and useful tool for measuring and forecasting the pandemic, perhaps THE most useful
- Bird 'flu remains a threat and spreads through farmed and wild animal populations, with more than half the Common Terns breeders dying off in the Dublin Port colony https://birdwatchireland.ie/devastating-bird-flu-impacts-on-irish-seabirds-revealed-in-new-study/

- https://github.com/dpnolan/pandemice

Modelling the COVID Pandemic in Ireland with Bayesian Methods in Python using PyMC and STAN

Peter Nolan
https://github.com/dpnolan/pandemic
databeaker@gmail.com
@dpnolan

Python Ireland
Sunday 16 November 2025
UCD